Laboratorium 1 - Podstawy WebGL2

Celem ćwiczenia jest zapoznanie się z podstawowym sposobem tworzenia grafiki trójwymiarowej w przeglądarce internetowej za pomocą WebGL2. W ramach laboratorium przygotowana zostanie statyczna scena gry w stylu Road Hopper, zawierająca drogę, trawę, gracza, samochody i drzewa. Obiekty zostaną zbudowane z wykorzystaniem geometrii płaszczyzny i sześcianu.

Podczas zajęć należy posługiwać się dołączoną paczką startową. Zawiera ona wstępną strukturę katalogów, geometrię i dane sceny. Samodzielnego uzupełnienia wymagają fragmenty odpowiedzialne za shadery, buforowanie wierzchołków i rysowanie. Miejsca te oznaczono komentarzami TODO 1–10

Przygotowanie środowiska

1. Uruchom Visual Studio Code i otwórz nowy terminal. Sprawdź czy w systemie zainstalowano Node.js oraz menedżer pakietów npm. W tym celu wykonaj poniższe polecenia:

```
node --version
npm --version
```

Jeżeli w obu przypadkach pojawią się numery wersji, przejdź do następnego kroku. Jeżeli system nie rozpoznaje polecenia node lub npm, przejdź na stronę https://nodejs.org/en/download, pobierz zalecaną wersję LTS dla systemu Windows i przeprowadź instalację z domyślnymi ustawieniami. Po zakończeniu instalacji zamknij i ponownie uruchom terminal Visual Studio Code, a następnie powtórz sprawdzenie wersji. Używana wersja Node.js musi być zgodna z wymaganiami zainstalowanej wersji Vite.

2. Przygotuj projekt obsługiwany przez Vite. Vite jest narzędziem uruchamiającym lokalny serwer deweloperski oraz obsługującym moduły JavaScript. Dzięki temu zmiany w plikach są szybko widoczne w przeglądarce. Pusty projekt tego typu można utworzyć poleceniem:

```
npm create vite@latest road-hopper-3d -- --template vanilla
```

Podczas tego laboratorium należy jednak skorzystać z dołączonej paczki **„Road_Hopper_3D_Lab1_STARTER.rar”**, która zawiera wymaganą strukturę projektu. Nie trzeba jednocześnie tworzyć pustego projektu poleceniem przedstawionym wyżej. Rozpakuj paczkę, a następnie otwórz katalog road-hopper-3d w Visual Studio Code.

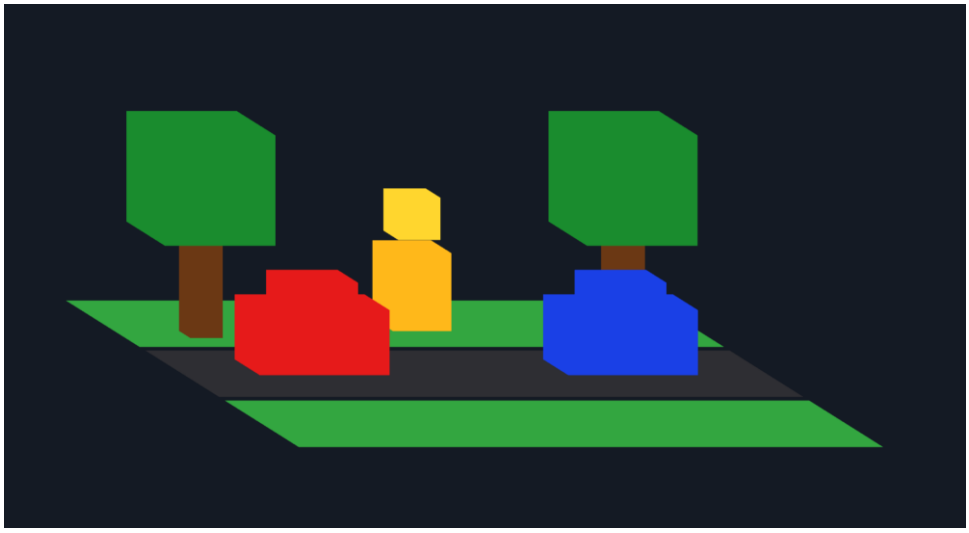
3. W terminalu otwartym w katalogu projektu wykonaj poniższe polecenia.

```
npm install
npm run dev
```

Pierwsza komenda pobiera zależności opisane w package.json, a druga uruchamia lokalny serwer Vite. Po uruchomieniu serwera w terminalu powinien pojawić się adres lokalny, najczęściej http://localhost:5173/. Otwórz ten adres w przeglądarce.

Pierwsze uruchomienie projektu

Po otwarciu adresu Vite obejrzyj gotową scenę. Powinny być widoczne dwa zielone pasy trawy rozdzielone szarą drogą, żółta postać, dwa samochody oraz dwa drzewa. Wszystkie elementy zbudowano z dwóch rodzajów geometrii: sześcianu oraz płaszczyzny. Nie poruszają się jeszcze, ponieważ w tym laboratorium przygotowujemy wyłącznie statyczny etap gry.



Jak WebGL2 tworzy obraz?

WebGL (Web Graphics Library) jest interfejsem pozwalającym aplikacji JavaScript korzystać z procesora graficznego (GPU) do wyświetlania grafiki. W niniejszym ćwiczeniu stosowany jest WebGL2, dostępny za pomocą elementu HTML canvas. WebGL nie udostępnia gotowego polecenia „narysuj samochód", dlatego konieczne jest przygotowanie parametrów oczekiwanego kształtu, przesłanie go do GPU, konfiguracja shaderów i wydanie poleceń jego rysowania na ekranie.



Pomiędzy Vertex Shaderem a Fragment Shaderem GPU składa trójkąty i wykonuje rasteryzację.

Schemat przedstawia kolejne etapy tworzenia obrazu w WebGL2. Najpierw **JavaScript** przygotowuje dane opisujące geometrię obiektu, np. współrzędne wierzchołków sześcianu. Dane te są przesyłane do bufora **VBO**, z którego może korzystać GPU. Obiekt **VAO** określa, w jaki sposób należy odczytywać zapisane liczby, np. po trzy wartości odpowiadające współrzędnym X, Y i Z. Następnie **Vertex Shader** oblicza pozycję każdego wierzchołka, uwzględniając np. przesunięcie i skalę obiektu. Na podstawie tych pozycji GPU tworzy trójkąty i przeprowadza rasteryzację, czyli ustala, które fragmenty obrazu są przez nie zajmowane. Na końcu **Fragment Shader** wyznacza kolor poszczególnych fragmentów, dzięki czemu powstaje obraz wyświetlany na ekranie.

Kod projektu został podzielony na moduły **ES (ECMAScript Modules)**. Modułami nazywamy pliki JavaScript, które mogą udostępniać funkcje i zmienne innym plikom. Instrukcja *export* udostępnia element, natomiast import umożliwia jego użycie. Na przykład funkcja createMesh jest udostępniona z pliku mesh.js i używana w main.js. Plik index.js znajdujący się w katalogu webgl zbiera kilka eksportów w jednym miejscu. Taki podział ułatwi późniejszą wymianę sposobu renderowania przy zachowaniu danych gry.

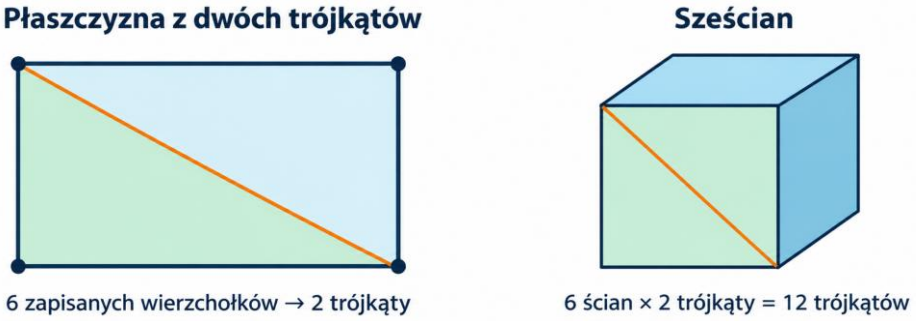
```
// W pliku mesh.js:
export function createMesh(gl, program, geometry) { /* ... */ }

// W innym module:
import { createMesh } from './webgl/mesh.js';
```

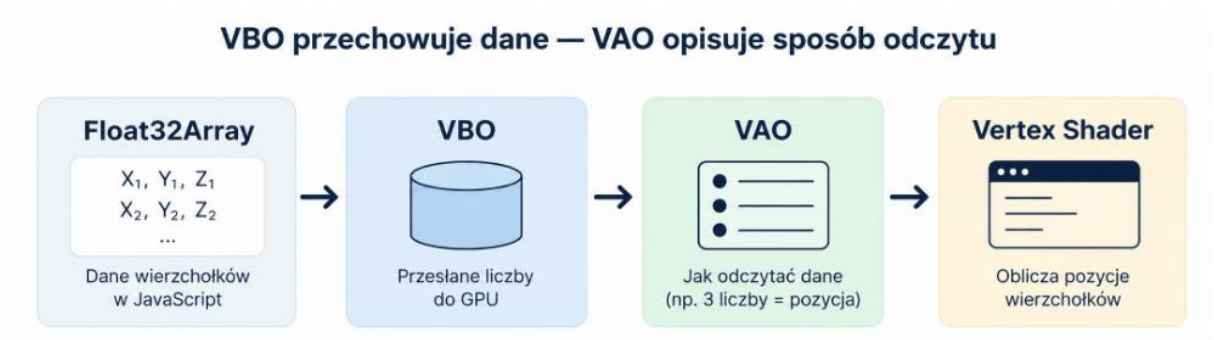
Przyjrzyj się strukturze projektu. Plik *src/main.js* odpowiada za uruchomienie WebGL i kolejność rysowania sceny. Katalog *src/game* przechowuje dane obiektów, katalog *src/webgl* zawiera funkcje rysujące oraz przygotowujące bufory, a katalog *src/shaders* przechowuje programy wykonywane przez GPU. Nie trzeba przepisywać tych plików; należy je otwierać i analizować podczas kolejnych kroków.

```
src/
├── game/
│   └── sceneData.js
├── shaders/
│   ├── vertexShader.js
│   ├── fragmentShader.js
│   └── shaderProgram.js
├── webgl/
│   ├── geometry.js
│   ├── mesh.js
│   ├── renderer.js
│   ├── primitiveObjects.js
│   └── index.js
├── main.js
└── style.css
```

Otwórz *src/webgl/geometry.js*. Zwróć uwagę na *planeGeometry* i *cubeGeometry*. WebGL potrafi rysować punkty, linie i trójkąty, ale nie posiada wbudowanej instrukcji tworzącej gotowy sześcian. Dlatego płaszczyzna składa się z dwóch trójkątów, a sześcian z dwunastu trójkątów. Każde trzy kolejne liczby zapisane w Float32Array to współrzędne jednego wierzchołka X, Y, Z.



W pliku *src/webgl/mesh.js* odszukaj *createBuffer()*, *bufferData()* i *vertexAttribPointer()*. *createBuffer()* tworzy VBO, *bufferData()* przesyła liczby do bufora, a *vertexAttribPointer()* opisuje ich interpretację. Jest to potrzebne, ponieważ VBO przechowuje liczby, ale sam nie wie, że trzy kolejne wartości oznaczają współrzędne X, Y i Z. Tę konfigurację zapamiętuje VAO. W czasie rysowania renderer.js wystarczy odwołać się do wcześniej przygotowanego VAO.



Shadery i uniformy

Shader to program napisany w języku GLSL (OpenGL Shading Language), wykonywany bezpośrednio przez procesor graficzny (GPU). Jego zadaniem jest określenie sposobu przetwarzania geometrii oraz wyglądu renderowanych obiektów.

W WebGL2 wykorzystujemy dwa podstawowe rodzaje shaderów:

- **Vertex Shader** przetwarza każdy wierzchołek obiektu, obliczając jego położenie na podstawie współrzędnych oraz dodatkowych parametrów, takich jak pozycja czy skala obiektu.
- **Fragment Shader** jest wykonywany po rasteryzacji trójkątów i określa kolor poszczególnych fragmentów powierzchni. W bardziej zaawansowanych zastosowaniach może również uwzględniać tekstury, oświetlenie i inne efekty wizualne.

Otwórz *src/shaders/vertexShader.js* i *src/shaders/fragmentShader.js*. Kod GLSL zapisany jest w postaci tekstu, ponieważ JavaScript nie wykonuje go bezpośrednio; WebGL przekazuje ten tekst do kompilatora shadera za pomocą *gl.shaderSource()*.

Uniform to parametr przekazywany z JavaScript do shaderów. W jednym poleceniu rysowania posiada wspólną wartość dla przetwarzanych wierzchołków lub fragmentów. Uniform *uPosition* określa pozycję obiektu, *uScale* jego rozmiar, a *uColor* kolor. Zmiana tych parametrów pozwala wykorzystać ten sam sześcian jako gracza, samochód lub drzewo. W *src/webgl/renderer.js* odszukaj wywołania *gl.uniform3fv()* i sprawdź, w jaki sposób wartości trafiają do shaderów.



Aktualny **Vertex Shader** wykorzystuje uproszczony rzut ukośny do pokazania sceny przestrzennej. W przyszłym laboratorium zastąpimy go macierzami Model (ustawienie obiektu), View (położenie i orientacja kamery) oraz Projection (rzutowanie przestrzeni na ekran). Na razie nie należy ich implementować.

Zadania do wykonania w trakcie laboratorium

Należy najpierw uruchomić projekt w wersji początkowej, a następnie wykonać dokładnie jedną opisaną zmianę. Po każdej zmianie należy zapisać plik, wrócić do przeglądarki i porównać wygląd sceny. Jeżeli Vite nie odświeży strony automatycznie, użyj skrótu klawiszowego **Ctrl+R**. Przed przejściem do kolejnego kroku przywróć pierwotną wartość! Podczas pracy zwróć uwagę na to, że modyfikacje wykonujemy w różnych warstwach projektu: danych gry, opisie wyglądu prymitywów, shaderach i poleceniach WebGL.

1. Otwórz *src/game/sceneData.js* i odszukaj pierwszy samochód w tablicy cars. Jego pozycja została zapisana jako [-0.45, 0.0, 0.0]. Zmień pierwszą wartość z -0.45 na -0.15:

Po zapisaniu pliku sprawdź, w którą stronę przesunął się czerwony samochód.

2. W tym samym obiekcie zmień kolor z [0.92, 0.12, 0.13] na [0.65, 0.20, 0.90] i ponownie sprawdź otrzymany efekt.

3. Otwórz *src/webgl/primitiveObjects.js*, znajdź funkcję *drawCar()* i pierwsze wywołanie *drawMesh()*, które odpowiada za nadwozie. Zmień jego skalę z [0.40, 0.20, 0.22] na [0.55, 0.20, 0.22]. Zauważ, że zmiana wpłynęła na wygląd obu pojazdów.

4. W funkcji *drawRow()* znajdź instrukcję wybierającą kolor na podstawie *row.type*. Zmień szary kolor drogi [0.19, 0.19, 0.21] na [0.36, 0.36, 0.42]. Sprawdź, czy zmieniła się wyłącznie droga. W tym przykładzie o wyborze koloru decyduje typ pasa przechowywany w *sceneData.js*, a za samo rysowanie nadal odpowiada ten sam *planeMesh*.

5. Otwórz *src/shaders/vertexShader.js*. W linii obliczającej *screenX* zmień współczynnik przy p.z z 0.35 na 0.15:

float screenX = p.x - p.z * 0.15;

Po zapisaniu pliku obejrzyj, jak zmieniło się nachylenie pasów podłoża i widok całej sceny. Nie przesuwałeś osobno żadnego samochodu ani drzewa; zmieniłeś regułę obliczania pozycji wszystkich wierzchołków. To pokazuje, dlaczego jeden Vertex Shader może wpływać na wszystkie obiekty. Przywróć wartość 0.35.

6. Otwórz *src/shaders/fragmentShader.js*. Zamiast przekazywać uniform *uColor* bez zmian, tymczasowo odejmij jego składowe od jedynki:

outColor = vec4(1.0 - uColor, 1.0);

Sprawdź, jak zmieniły się kolory gracza, samochodów, drzew i podłoża. Fragment Shader wykonuje tę samą operację dla fragmentów wszystkich rysowanych obiektów. Zwróć uwagę, że kolor tła ustawiany przez *gl.clearColor()* nie podlega temu przekształceniu. Po eksperymencie przywróć *outColor = vec4(uColor, 1.0);*.

7. Otwórz *src/webgl/renderer.js*. W funkcji *drawMesh()* zamień na chwilę *gl.TRIANGLES* na *gl.LINES*:

gl.drawArrays(gl.LINES, 0, mesh.vertexCount);

Przy trybie LINES WebGL łączy wierzchołki parami, podczas gdy TRIANGLES łączy je w trójkąty. Nie jest to prawidłowy sposób otrzymania pełnej siatki krawędzi sześcianu, lecz prosty eksperyment pokazujący znaczenie trybu rysowania. Koniecznie przywróć gl.TRIANGLES.

8. W *src/main.js* znajdź wywołanie *gl.enable(gl.DEPTH_TEST)* w funkcji *render()*. Zmień je tymczasowo na *gl.disable(gl.DEPTH_TEST)*, a następnie sprawdź, czy niektóre części sceny zaczęły się wzajemnie zasłaniać w niewłaściwej kolejności. Bez testu głębokości większe znaczenie ma kolejność wysyłania poleceń rysowania. Skutek może być różny w zależności od tego, które bryły nachodzą na siebie w aktualnym widoku. Po zakończeniu przywróć *gl.enable(gl.DEPTH_TEST).*

Zadania do sprawozdania

Utwórz plik sprawozdania. Zapisuj w nich efekty zadań wykonanych podczas zajęć oraz poniższych zajęć z WebGL2. Jako efekt pracy na laboratoriach, konieczne będzie przesłanie pojedynczego zbiorczego sprawozdania z zadań wykonanych na trzech laboratoriach.

Zadanie 1. Dodatkowy pojazd.

Dodaj trzeci samochód o unikalnym kolorze i pozycji, tak aby wszystkie trzy pojazdy były widoczne na drodze. Nie twórz nowej geometrii ani osobnej funkcji rysującej.

Zadanie 2. Zalesianie sceny.

Dodaj trzecie drzewo na drugim pasie trawy. Nowe drzewo powinno mieć wyższy pień i czerwoną, jesienną koronę, natomiast pozostałe drzewa powinny zachować dotychczasowy wygląd.

Zadanie 3. Dodatkowy pas planszy.

Rozbuduj planszę o czwarty pas trawy, znajdujący się na pozycji z: -1.40. Nadaj mu kolor różniący się od pozostałych pasów. Nie twórz nowej geometrii.

Zadanie 4. Zmiana perspektywy

Zmień sposób wyświetlania sceny tak, aby była obserwowana pod innym kątem. Zadbaj o zachowanie widoczności wszystkich obiektów. W sprawozdaniu opisz wprowadzone zmiany i przedstaw porównanie widoku przed i po modyfikacji.

Wymagania dotyczące sprawozdania

Sprawozdanie powinno być podzielone na poszczególne laboratoria, z wyraźnym oznaczeniem **numeru laboratorium oraz numerów wykonanych zadań**.

Dla każdego zadania należy zamieścić:

1. **Opis rozwiązania** - krótkie wyjaśnienie, w jaki sposób wykonano zadanie, np. poprzez przedstawienie zmodyfikowanego fragmentu kodu wraz z krótkim komentarzem.
2. **Zrzut ekranu**- przedstawiający efekt wprowadzonych zmian na renderowanej scenie.

Po zakończeniu trzech laboratoriów z WebGL2 należy przesłać **jedno zbiorcze sprawozdanie**, zawierające rozwiązania zadań ze wszystkich trzech spotkań.